Helpdesk Ticketing App

Aplikasi helpdesk ticketing berbasis web untuk mengelola tiket bantuan teknis, dibangun dengan Laravel 13 + React 19 (Inertia.js) + Tailwind CSS 4.

Daftar Isi

- Persyaratan Sistem
- Quick Start (Docker)
- Arsitektur Docker
- Perintah Makefile
- Perintah Artisan dalam Container
- Service & Port
- Akun Default
- Instalasi Manual (tanpa Docker)
- Struktur Halaman
- Fitur Utama
- Alur Perizinan (Policy)
- Database Schema
- Konfigurasi Lanjutan
- Performa & Optimasi
- Troubleshooting

Persyaratan Sistem

Dengan Docker

Kebutuhan	Minimal
Docker	^24.0

Kebutuhan	Minimal
Docker Compose	^2.20
RAM	2 GB
Disk	5 GB

Tanpa Docker

Kebutuhan	Versi
PHP	^8.3
Composer	^2.0
Node.js	^18.0
NPM	^9.0
Database	MySQL ^8.0 / PostgreSQL ^16 / SQLite 3
Ekstensi PHP	pdo_mysql / pdo_pgsql / pdo_sqlite, mbstring, xml, bcmath, curl, zip, redis

---

Quick Start (Docker)

1. Clone Repository

```
git clone <repository-url> ticketing-app
cd ticketing-app
```

2. Jalankan Semua Service

```
docker compose up -d
```

Tunggu hingga service siap (sekitar 60-90 detik pertama kali). Entrypoint otomatis akan:

1. Copy `.env.docker` → `.env`
2. Buat folder `storage/` dan `bootstrap/cache/`
3. Install dependensi PHP (`composer install`)
4. Generate `APP_KEY`
5. Buat storage symlink
6. Tunggu database siap → jalankan `migrate`
7. Seed database (role, user, kategori)
8. Warm up cache

3. Build Frontend Assets

```
npm install
npm run build
```

> **PENTING**: Pastikan tidak ada file `public/hot`. Jika ada, hapus dengan `rm public/hot` agar aplikasi menggunakan aset build.

4. Buka Browser

```
http://localhost:8000
```

5. Login

Email	Password	Role
`admin@helpdesk.com`	`password`	Admin
`staff@helpdesk.com`	`password`	Staff
`user@helpdesk.com`	`password`	Customer

Arsitektur Docker

```
┌──────────────────────────────────────────────────────┐
│                    Docker Host                       │
│                                                      │
│  ┌──────────┐    ┌──────────┐    ┌──────────┐       │
│  │  Nginx   │──▶│ PHP-FPM  │──▶│  MySQL   │        │
│  │  :8000   │    │  (app)   │    │  :3306   │       │
│  └──────────┘    └──────┬───┘    └──────────┘       │
│                         │                            │
│                         ├──▶ ┌──────────┐           │
│                         │    │  Redis   │           │
│                         │    │  :6379   │           │
│                         │    └──────────┘           │
│                         │                            │
│                         ├──▶ ┌──────────┐           │
│                         │    │ MailHog  │           │
│                         │    │  :8025   │           │
│                         │    └──────────┘           │
│                         │                            │
│  ┌──────────┐           │                            │
│  │  Queue   │──────────┘                            │
│  │  Worker  │                                       │
│  └──────────┘           ┌──────────┐                │
│                         │ Adminer  │                │
│                         │  :8080   │                │
│                         └──────────┘                │
└──────────────────────────────────────────────────────┘
```

Detail Service

Service	Container	Image	Port	Fungsi
app	`helpdesk-app`	`php:8.3-fpm-alpine` (built)	internal	Application server, handle request PHP
nginx	`helpdesk-nginx`	`nginx:1.26-alpine`	`8000→80`	Web server, reverse proxy, static assets
db	`helpdesk-db`	`mysql:8.0`	`3306`	Database MySQL 8.0
redis	`helpdesk-redis`	`redis:7-alpine`	`6379`	Cache & session driver
queue	`helpdesk-queue`	`php:8.3-fpm-alpine` (built)	internal	Queue worker untuk notifikasi async
mailhog	`helpdesk-mailhog`	`mailhog/mailhog:v1.0.1`	`8025→8025`	Email testing (SMTP di port `1025`)
adminer	`helpdesk-adminer`	`adminer:4.8.1-standalone`	`8080→8080`	Database management UI
vite	`helpdesk-vite`	`node:22-alpine`	`5173→5173`	Vite dev server

Service	Container	Image	Port	Fungsi
				(hanya profile `dev`)

Volume Persistent

Volume	Path	Isi
`db-data`	`/var/lib/mysql`	Data MySQL
`db-backups`	`/backups`	Backup database
`redis-data`	`/data`	Data Redis (RDB + AOF)
`app-storage`	`/var/www/html/storage`	Log, cache, uploads
`app-vendor`	`/var/www/html/vendor`	Dependensi Composer
`vite-node-modules`	`/var/www/html/node_modules`	Dependensi NPM (dev)
`nginx-logs`	`/var/log/nginx`	Log Nginx

Network

Semua service berkomunikasi melalui network bridge `app-network`. Service saling menjangkau menggunakan hostname container:

- `app` → PHP-FPM di port 9000
- `db` → MySQL di port 3306
- `redis` → Redis di port 6379
- `mailhog` → SMTP di port 1025

Healthcheck

Setiap service memiliki healthcheck untuk memastikan dependensi berjalan dengan urutan yang benar:

```
db (healthy) ──┐
               ├──▶ app (healthy) ──▶ nginx
redis (healthy)┘        │
                        └──▶ queue
```

Perintah Makefile

Gunakan `make` untuk shortcut perintah yang paling sering dipakai.

Start / Stop

```
make up              # Start semua service (background)
make down            # Stop semua service
make restart         # Restart semua service
make build           # Build ulang image (no cache)
make rebuild         # Down → build → up
```

Development

```
make dev             # Start service + Vite hot-reload
make shell           # Masuk shell container app
make mysql           # Masuk MySQL shell
make redis-cli       # Masuk Redis CLI
```

Database

```
make migrate         # Jalankan migrasi
make seed            # Jalankan seeder
make fresh           # Reset database + seed
```

Artisan

```
make art t=migrate:status    # Jalankan artisan apa saja
make tinker                   # Masuk Laravel Tinker
make cache                    # Warm up cache (optimize)
make clear                    # Clear semua cache
```

Monitoring

```
make logs            # Tail semua log service
make ps              # Status container
make stats           # Resource usage container
```

Cleanup

```
make prune           # Hapus semua container + volume + image
```

Perintah Artisan dalam Container

Migrasi & Seeder

```
docker compose exec app php artisan migrate              # Jalankan migrasi
docker compose exec app php artisan migrate:rollback     # Rollback migrasi
docker compose exec app php artisan migrate:fresh --seed # Reset DB + seed
docker compose exec app php artisan db:seed              # Seed database
```

Cache

```
docker compose exec app php artisan optimize             # Warm up cache (confi
docker compose exec app php artisan optimize:clear       # Clear semua cache
docker compose exec app php artisan config:clear         # Clear config cache
docker compose exec app php artisan route:clear          # Clear route cache
docker compose exec app php artisan view:clear           # Clear view cache
```

Storage

```
docker compose exec app php artisan storage:link         # Buat symlink storage
docker compose exec app php artisan storage:unlink       # Hapus symlink storag
```

Queue

```
docker compose exec app php artisan queue:failed         # Lihat failed jobs
docker compose exec app php artisan queue:retry all      # Retry semua failed j
docker compose exec app php artisan queue:flush          # Hapus semua failed j
docker compose restart queue                             # Restart queue worker
```

Testing

```
docker compose exec app php artisan test                 # Jalankan PHPUnit
docker compose exec app php artisan test --filter=...    # Filter test spesifik
```

Log & Monitoring

```
docker compose logs -f app          # Log PHP-FPM
docker compose logs -f nginx        # Log Nginx
docker compose logs -f queue        # Log queue worker
docker compose logs -f db           # Log MySQL
docker compose logs -f --tail=100   # 100 baris terakhir semua
```

Service & Port

Aplikasi Utama

URL	Keterangan
`http://localhost:8000`	Aplikasi Helpdesk

URL	Keterangan
`http://localhost:8000/health`	Health check Nginx

Email Testing (MailHog)

URL	Keterangan
`http://localhost:8025`	MailHog Web UI
SMTP: `localhost:1025`	SMTP endpoint

Cara test email: Semua email yang dikirim aplikasi (misal: notifikasi, reset password) otomatis tertangkap MailHog. Buka `http://localhost:8025` untuk melihatnya. Tidak ada email yang benar-benar terkirim ke luar.

Database Management (Adminer)

URL	Keterangan
`http://localhost:8080`	Adminer UI

Login Adminer:

- **System**: MySQL
- **Server**: `db`
- **Username**: `helpdesk`
- **Password**: `secret`
- **Database**: `helpdesk_db`

Vite Dev Server (Hot Reload)

```
docker compose --profile dev up -d vite
```

URL	Keterangan
`http://localhost:5173`	Vite HMR server

Saat Vite berjalan, file `public/hot` otomatis dibuat. CSS/JS React dimuat via hot-reload.

Untuk kembali ke aset build, hentikan Vite dan hapus file hot:

```
docker compose --profile dev down vite
rm public/hot
```

Akun Default

Setelah `db:seed`, 3 akun tersedia:

Email	Password	Role	Akses
`admin@helpdesk.com`	`password`	admin	Dashboard Admin, Kelola Tiket, Kelola User, Kelola Kategori, SAW Config
`staff@helpdesk.com`	`password`	staff	Dashboard Admin, Kelola Tiket, Komentar Internal
`user@helpdesk.com`	`password`	customer	Portal Pelanggan, Buat Tiket, Lihat Tiket Sendiri

Role & Permission

Role	Permission
admin	Semua akses — full CRUD tiket, user, kategori, knowledge base, SAW config

Role	Permission
staff	Kelola tiket, assign tiket, komentar internal, lihat dashboard admin
customer	Buat tiket, lihat tiket sendiri, komentar publik, batalkan tiket, beri rating

Data Awal

- **3 role** — admin, staff, customer
- **5 kategori** — Hardware, Software, Jaringan, Akses Akun, Lainnya

---

Instalasi Manual (tanpa Docker)

1. Clone & Install Dependensi

```
git clone <repository-url> ticketing-app
cd ticketing-app
composer install
cp .env.example .env
php artisan key:generate
```

2. Konfigurasi Database

Edit `.env` — pilih salah satu:

▶ **MySQL / MariaDB**
▶ **PostgreSQL**
▶ **SQLite** (zero-config, cocok untuk dev)

3. Konfigurasi Session & Cache

Untuk development tanpa Redis, gunakan file driver:

```
SESSION_DRIVER=file
CACHE_STORE=file
QUEUE_CONNECTION=sync
```

4. Migrasi, Seed, Storage

```
php artisan migrate
php artisan db:seed
php artisan storage:link
```

5. Install & Build Frontend

```
npm install
npm run build
```

6. Jalankan Aplikasi

Development (dengan hot reload):

```
composer dev
```

Menjalankan 4 proses sekaligus: `php artisan serve`, `php artisan queue:listen`, `php artisan pail`, `npm run dev`

Manual (tanpa Composer script):

```
php artisan serve                    # Terminal 1 — backend
php artisan queue:listen --tries=1   # Terminal 2 — queue worker
npm run dev                          # Terminal 3 — Vite HMR
```

Buka browser di `http://127.0.0.1:8000`

Struktur Halaman

Portal Pelanggan (role: customer)

Route	Halaman	Fungsi
`/portal/dashboard`	Dashboard	Ringkasan tiket pelanggan
`/portal/tickets`	Daftar Tiket	Lihat semua tiket milik pelanggan
`/portal/tickets/create`	Buat Tiket	Form pembuatan tiket + upload lampiran + kamera
`/portal/tickets/{id}`	Detail Tiket	Lihat detail, komentar, batalkan, beri rating
`/portal/knowledge-base`	Basis Pengetahuan	Artikel-artikel bantuan
`/portal/knowledge-base/{id}`	Artikel	Detail artikel bantuan
`/notifications`	Notifikasi	Riwayat notifikasi aktivitas tiket

Admin Panel (role: staff, admin)

Route	Halaman	Fungsi
`/admin/dashboard`	Dashboard Admin	Statistik, chart, tiket terbaru, beban kerja staff
`/admin/tickets`	Kelola Tiket	Daftar semua tiket + filter (status, prioritas, kategori, petugas)
`/admin/tickets/{id}`	Detail Tiket	Ubah status/prioritas/penugasan, komentar internal, log aktivitas

Route	Halaman	Fungsi
`/admin/users`	Kelola Pengguna	CRUD user + assign role
`/admin/categories`	Kelola Kategori	CRUD kategori tiket
`/admin/knowledge-base`	Pusat Bantuan	CRUD artikel basis pengetahuan
`/admin/templates`	Template Respon	CRUD template respons cepat untuk komentar
`/admin/saw-configuration`	Konfigurasi SAW	Atur bobot kriteria prioritas otomatis

Fitur Utama

Tiket Helpdesk

- **Status Flow** — `open` → `in_progress` → `resolved` → (user confirm) → `closed`. Tiket juga bisa `cancelled`.
- **Konfirmasi Ganda** — Saat admin set `resolved`, user harus konfirmasi: **Ya, Selesai** (→ `closed`) atau **Belum, Masih Ada Masalah** (→ `in_progress`).
- **Prioritas** — `low`, `medium`, `high`, `critical` dengan badge warna + skor SAW otomatis.
- **Penugasan** — Admin menugaskan staff ke tiket.
- **Komentar Internal** — Staff/admin bisa menambahkan catatan internal yang tidak terlihat pelanggan.
- **Komentar Terkunci** — Otomatis dikunci saat tiket `closed` atau `cancelled`.

- **Template Respons** — Admin bisa buat template pesan cepat untuk komentar.
- **Rating** — Pelanggan bisa memberi rating (1-5 bintang) setelah tiket selesai.
- **Lampiran** — Upload file (foto, PDF) saat buat tiket atau komentar. Bisa ambil foto langsung dari kamera.
- **Pembatalan** — Pelanggan bisa membatalkan tiket yang masih aktif.

Notifikasi

- Notifikasi otomatis dikirim ke pelapor dan petugas saat tiket dibuat, diperbarui, atau dikomentari.
- Disimpan di database, ditampilkan di halaman notifikasi dengan badge unread.
- Admin bisa menandai semua notifikasi sebagai dibaca.

Dashboard Admin

- Statistik total, open, in progress, resolved, closed, overdue
- Chart distribusi status dan prioritas (Doughnut Chart)
- Daftar tiket terbaru
- Beban kerja staff (jumlah tiket yang ditugaskan)

Basis Pengetahuan

- Artikel bisa berstatus draft atau publish
- Hanya artikel publish yang terlihat pelanggan
- Dikelompokkan per kategori

Alur Perizinan (Policy)

TicketPolicy

Aksi	customer	staff	admin
Lihat tiket sendiri (`view`)	✅	✅	✅
Lihat semua tiket	❌	✅	✅
Buat tiket (`create`)	✅	✅	✅
Update tiket (`update`)	Hanya tiket sendiri yang masih aktif	✅	✅
Hapus tiket (`delete`)	Hanya tiket sendiri yang open/cancelled	✅	✅
Komentar (`comment`)	Hanya tiket sendiri	✅	✅
Batalkan (`cancel`)	Hanya tiket sendiri yang masih aktif	✅	✅

Field yang Bisa Diubah Admin di Tiket

Field	Bisa diubah admin?
Status	✅
Prioritas	✅
Ditugaskan ke (assigned_to)	✅
Judul	❌ (data asli pelanggan)
Deskripsi	❌ (data asli pelanggan)
Kategori	❌ (data asli pelanggan)

Role Middleware

Route Group	Middleware
Portal (dashboard, buat tiket)	`auth` + `role:customer`
Portal (lihat, komentar, rating)	`auth` + `role:customer|staff|admin`
Admin (semua)	`auth` + `role:staff|admin`
Notifikasi	`auth`

Database Schema

Tabel Utama

```
users
├── id, name, email, password, phone, department
├── timestamps
└── (pivot: model_has_roles → roles)

tickets
├── id, uuid (unique), user_id (FK), category_id (FK)
├── title, description, priority, status
├── assigned_to (FK users, nullable), sla_deadline
├── resolved_at, resolved_confirmed_at, cancelled_at
├── rating, rating_comment
└── timestamps

comments
├── id, ticket_id (FK), user_id (FK)
├── message, is_internal, attachments (JSON)
└── timestamps

categories
├── id, name, slug (unique), description
└── timestamps

knowledge_bases
├── id, title, slug (unique), content (longText)
```

```
├── category_id (FK, nullable), author_id (FK)
├── is_published (boolean)
└── timestamps

ticket_templates
├── id, title, content, category_id (FK, nullable)
├── created_by (FK users)
└── timestamps

activity_logs
├── id, user_id (FK, nullable), ticket_id (FK, nullable)
├── action, description
└── timestamps

notifications
├── id (UUID), type, notifiable_type, notifiable_id
├── data (JSON), read_at
└── timestamps

saw_configurations
├── id, code (unique), name, type (benefit/cost)
├── weight (decimal), sort_order
└── timestamps

personal_access_tokens
├── id, tokenable_type, tokenable_id
├── name, token, abilities (JSON)
├── last_used_at, expires_at
└── timestamps
```

Enum Values

Kolom	Values
`tickets.priority`	`low` , `medium` , `high` , `critical`
`tickets.status`	`open` , `in_progress` , `resolved` , `closed` , `cancelled`

Kolom	Values
`comments.is_internal`	`0` (publik), `1` (internal)
`knowledge_bases.is_published`	`0` (draft), `1` (publish)

Konfigurasi Lanjutan

Environment Variables (`.env.docker`)

```
# Aplikasi
APP_NAME=Helpdesk
APP_ENV=local                          # local | production
APP_DEBUG=true                         # true | false
APP_URL=http://localhost:8000

# Database (Docker)
DB_CONNECTION=mysql
DB_HOST=db
DB_PORT=3306
DB_DATABASE=helpdesk_db
DB_USERNAME=helpdesk
DB_PASSWORD=secret

# Redis (Cache + Session + Queue)
REDIS_HOST=redis
REDIS_PORT=6379
REDIS_PASSWORD=null
REDIS_CLIENT=phpredis
SESSION_DRIVER=redis
CACHE_STORE=redis
QUEUE_CONNECTION=redis

# Mail (MailHog untuk development)
MAIL_MAILER=smtp
MAIL_HOST=mailhog
MAIL_PORT=1025
```

Deploy ke Server Produksi

```
# 1. Clone di server
git clone <url> /opt/helpdesk
cd /opt/helpdesk

# 2. Copy & edit environment
cp .env.docker .env
# Edit APP_ENV=production, APP_DEBUG=false, APP_URL=<domain>

# 3. Build frontend di host (sebelum Docker)
npm install
npm run build

# 4. Start service
docker compose up -d --build

# 5. Setup cron untuk Laravel scheduler
echo "* * * * * docker compose -f /opt/helpdesk/docker-compose.yml exec -T app p
```

Backup Database

```
# Backup
docker compose exec db mysqldump -u helpdesk -psecret helpdesk_db > backup_$(dat

# Restore
docker compose exec -T db mysql -u helpdesk -psecret helpdesk_db < backup_20250
```

Custom Port

Edit `docker-compose.override.yml`:

```
services:
  nginx:
    ports:
      - "8080:80"    # Ganti dari 8000 ke 8080
  db:
```

```
    ports:
      - "3307:3306"  # Ganti port MySQL
```

Performa & Optimasi

Optimasi	Keterangan
OPcache	256MB memory, 10000 max files, CLI enabled
APCu	Object caching via PECL
DB Indexes	`tickets(status, priority, user_id, assigned_to)`, `comments(ticket_id)`, `notifications(notifiable_id, read_at)`
Cache SAW	Skor SAW di-cache 60 detik via Redis
Cache Dashboard	Stats dashboard admin di-cache 300 detik
Debounce Search	Input search ditunda 400ms — kurangi request server
N+1 Prevention	Semua list pakai `->with()` eager loading
Nginx Gzip	Compression level 6 untuk text assets
Nginx Static	Cache 7 hari untuk CSS/JS/font/images
MySQL Tuning	InnoDB buffer pool 256MB, max connections 100
Redis Tuning	Max memory 256MB, LRU eviction, AOF persistence
FastCGI Tuning	Connection timeout 60s, buffer 16x16k, keepalive 8

Troubleshooting

Docker

Masalah	Solusi
Container tidak start	`docker compose logs app` — cek error
Database connection refused	Tunggu `db` healthcheck selesai (bisa 30-60s pertama kali)
Port 8000 sudah dipakai	Ubah port di `docker-compose.override.yml`
composer install gagal	`docker compose exec app composer install --ignore-platform-reqs`
Permission storage error	`docker compose exec app chmod -R 775 storage bootstrap/cache`
app_key kosong	`docker compose exec app php artisan key:generate`
Queue worker tidak jalan	`docker compose restart queue`
Redis connection error	`docker compose restart redis`

Aplikasi

Masalah	Solusi
Halaman putih / blank	Hapus `public/hot` lalu `npm run build` dan `php artisan optimize:clear`
CSS tidak muncul	`npm run build` lalu `php artisan optimize:clear`

Masalah	Solusi
Asset mengarah ke `localhost:5173`	Hapus file `public/hot` agar gunakan aset build
Upload lampiran gagal	`docker compose exec app php artisan storage:link`
Login error 419	`docker compose exec app php artisan optimize:clear`
Notifikasi tidak muncul	`docker compose restart queue`
Migration error	`docker compose exec app php artisan migrate:fresh --seed`

Database

Masalah	Solusi
Lupa password MySQL	Default: `helpdesk` / `secret` (lihat `docker-compose.yml`)
Reset database	`make fresh` atau `docker compose exec app php artisan migrate:fresh --seed`
Akses Adminer	Buka `http://localhost:8080` , Server: `db` , User: `helpdesk` , Pass: `secret`
Koneksi dari host	Host: `127.0.0.1` , Port: `3306` , User: `helpdesk` , Pass: `secret`